FIRST WORKING MODEL

People Counting with YOLO11 and ByteTrack

0.6548	1.7021	0.4113	0.4324
Mean counting accuracy	Curve MAE	MOTA	IDF1

Repository

https://github.com/NikolayTaran/cv_people_counting

Project objective

Build a complete pipeline that counts people entering and leaving a region, and measure its robustness under occlusion, crowding, and different camera views. The submitted system combines a YOLO11 person detector, a custom ByteTrack tracker, and a directional virtual-line counter.

Implemented architecture

Stage	Implementation	Output
Detection	YOLO11s, person class, confidence floor 0.05	Boxes and confidence
Tracking	Custom ByteTrack: Kalman + two-stage Hungarian IoU matching	Persistent track IDs
Counting	Foot-point crossing, hysteresis, cooldown, gap handling	Directional IN / OUT events
Evaluation	Same geometry for predicted and GT trajectories	CA, curve MAE, MOTA, IDF1

1. Data loading and preprocessing

MOTChallenge image sequences, seqinfo metadata, and gt/gt.txt annotations are loaded in frame order. Only valid class-1 pedestrian annotations are retained. Boxes are clipped to the image, boxes smaller than 2 px are removed, and training annotations with visibility below 0.10 are omitted. Remaining boxes are converted to normalized YOLO labels.

Purpose	Sequences	Images / frames	Boxes
Train	First 80% of MOT17-04 and MOT17-05	1,510	39,144
Validation	Final 20% of MOT17-04 and MOT17-05	377	11,397
Test	MOT17-02, MOT20-01, MOT17-09, MOT17-11	2,454	Never exported

The chronological development split reduces mixing of nearly identical adjacent frames. More importantly, the four evaluation cameras never enter the training exporter. VisDrone is configured as an optional aerial test, but it is omitted from the reported metrics because that sequence was unavailable through the reproducible download used for this run.

Preprocessing and augmentation

- Training input: 640 px letterbox; inference: 1280 px for small/dense scenes and 640 px for the other scenes.
- Horizontal flip, translation, scale jitter, light mixup, and mosaic augmentation.
- The detector sees only person boxes; tracking and counting targets are never used in detector optimization.

2. Model and training setup

YOLO11s was initialized from COCO weights and fine-tuned with AdamW. The run used lr0=0.001, cosine decay, weight decay 0.0005, batch size 8, AMP, seed 42, deterministic mode, and an NVIDIA GeForce RTX 4060 Laptop GPU. PyTorch 2.11.0+cu128 and Ultralytics 8.4.175 were used.

Checkpoint	Precision	Recall	mAP50	mAP50-95
3 epochs (selected)	0.9712	0.8911	0.9636	0.6781
20 epochs	0.9644	0.9147	0.9643	0.7246

3. Quantitative results

The primary model-selection metric is mean counting accuracy (CA). All candidates use the same tracker, counting geometry, and camera-separated evaluation set. Higher CA, MOTA, and IDF1 are better; lower curve MAE is better.

Detector	Mean CA	Curve MAE	MOTA	IDF1
YOLO11n, COCO pretrained	0.5501	2.5913	0.4361	0.4446
YOLO11s, COCO pretrained	0.6222	1.9747	0.4352	0.4671
YOLO11s, MOT fine-tuned 3 epochs	0.6548	1.7021	0.4113	0.4324
YOLO11s, MOT fine-tuned 20 epochs	0.5794	2.1542	0.4080	0.4277

The selected 3-epoch model improves mean CA by 0.1047 absolute over the original YOLO11n baseline and by 0.0326 over the larger pretrained baseline. The 20-epoch checkpoint obtains stronger validation detection metrics and improves dense MOT20-01, but it transfers less reliably to the other camera views.

Selected model by sequence

Sequence	Pred IN/OUT	GT IN/OUT	CA	Curve MAE	MOTA	IDF1
MOT17-02	10 / 1	16 / 5	0.5238	2.2500	0.1797	0.2997
MOT20-01	7 / 2	21 / 5	0.3462	2.8750	0.4700	0.4433
MOT17-09	9 / 3	10 / 3	0.9231	0.1000	0.4770	0.4535
MOT17-11	9 / 10	11 / 12	0.8261	1.5833	0.5185	0.5331

Interpretation

- The short fine-tune produces the best final counting score, especially on MOT17-09 and MOT17-11.
- The 20-epoch model raises MOT20-01 CA from 0.4231 to 0.6154, confirming that domain adaptation helps dense crowds.
- Detection mAP and end-to-end counting quality are related but not interchangeable; identity continuity near the counting line remains decisive.

4. Problems and observations

- A 20-epoch run initially failed inside OpenCV warpAffine when Windows data-loader worker processes were active. Training resumed from last.pt with workers=0 and completed normally. The stable reproduction command uses this setting.
- High development validation mAP did not guarantee the best camera-separated counting score. The validation frames come from the same two camera views as training.
- The tracker uses IoU and motion without an appearance/ReID branch. Long occlusions can fragment or switch identities even when detector recall is high.
- Counting geometry is configured per camera. Deployment on arbitrary video would need camera calibration or an automatic entrance-region estimator.

5. Reproduction

The repository contains the data exporter, deterministic training script, selected checkpoint, split-aware inference and evaluation CLIs, machine-readable reports, and 39 passing unit tests. The key commands are:

```
python scripts/prepare_training_data.py
```

```
python scripts/train_detector.py --epochs 3 --imgsz 640 --batch 8 --workers 0 --device 0
```

```
python scripts/run_pipeline.py --split test --weights models/people_yolo11s_mot.pt --device 0 --no-render --out-dir outputs/submission/runs
```

```
python scripts/evaluate.py --split test --runs-dir outputs/submission/runs --results outputs/submission/test_results.json
```

```
python -m pytest tests -q
```

Submission artifacts

- Selected model: models/people_yolo11s_mot.pt
- Full report: docs/FIRST_WORKING_MODEL.md
- Machine-readable results and training histories: reports/
- Repository: https://github.com/NikolayTaran/cv_people_counting